JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA-128

Department Of Computer Science Engineering And Information Technology

PROJECT TITLE: "MOVIELENS"

Under the supervision of:

Dr. Aditi Sharma

Name(s)	Enrollment no.(s)
Arjun Pratap Aggarwal	9922103025
Vansh Mittal	9922103219
Divyanshu Gupta	9922103223

Course Name: Big Data with Hadoop and Spark

Course Code: 21B2CS411

Program: B.Tech, CSE

4^{th} Year, 7th Sem

Date: November 24, 2025

(I)

TABLE OF CONTENTS

.

(II)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the faculty members who have played an instrumental role in the successful completion of this project. Their unwavering support, expertise, and encouragement have been pivotal throughout this journey. First and foremost, we extend our heartfelt thanks to **Dr. Aditi Sharma** for providing us with this incredible opportunity to work on this project. Her guidance, constructive feedback, and insightful suggestions have been invaluable in helping us navigate challenges and achieve our objectives.

We would also like to thank all the other faculty members of the department who, directly or indirectly, supported us during the project. Their lectures, discussions, and shared experiences provided us with the foundational knowledge and tools to approach this project with confidence. This project has been a profound learning experience, and we are deeply grateful to our faculty for not only imparting technical knowledge but also encouraging us to develop a problem-solving mindset, teamwork abilities, and an appreciation for continuous learning. Thank you for inspiring and guiding us throughout this enriching journey.

INTRODUCTION

In an era defined by rapidly growing data volumes, the domain of Big Data has emerged as a critical field for organizations aiming to extract meaningful insights and make data-driven decisions. The traditional data processing tools struggle when confronted with the three "V"s of Big Data i.e; Volume, Velocity, and Variety. To address these challenges, distributed storage and processing frameworks have gained prominence. One such ecosystem, Apache Hadoop, offers scalable, fault-tolerant infrastructure capable of processing massive datasets across clusters of commodity hardware.

This project, hosted in the Hadoop-Project repository, presents a comprehensive implementation of the Hadoop ecosystem to analyze the MovieLens 100k Dataset. The goal is not merely to demonstrate each tool in isolation, but to build a data pipeline that ingests, processes and queries the dataset using multiple Hadoop ecosystem components: distributed file storage via HDFS (Hadoop Distributed File System), data processing using MapReduce, and high-level query engines such as Hive.

By systematically integrating these components, this project aims to:

- Showcase how each component contributes to solving real-world Big Data processing tasks (e.g., data ingestion, transformation, querying).
- Provide insights into their performance and usability within a single data-pipeline context.
- Offer a replicable architecture and methodology for students and practitioners to understand and deploy Big Data solutions using the Hadoop stack.

In doing so, the project emphasises the architectural design of a Big Data workflow: starting from storing raw data in HDFS, performing transformations and analytics using MapReduce and Hive. This layered approach demonstrates the strengths of distributed processing frameworks in terms of scalability, efficiency, and suitability for large-scale analytical tasks.

OBJECTIVES

The primary objective of this project is to develop an end-to-end Big Data processing pipeline using the Apache Hadoop ecosystem to analyze the MovieLens dataset and derive meaningful insights from large-scale data. The project seeks to demonstrate the practical capabilities of distributed data processing frameworks by implementing real-world analytical use-cases using core Hadoop components. By building an operational pipeline, the project aims to illustrate how Hadoop enables reliable storage, parallel computation, and query-based data exploration in a scalable environment.

Specific Objectives:

- To store and manage data efficiently using HDFS, enabling scalable, fault-tolerant, and distributed storage that supports parallel access and reliability even in case of node failures.
- To process and analyze large datasets using MapReduce, demonstrating batch-processing capabilities that divide workloads into smaller tasks executed across multiple nodes for faster computation and aggregation of results.
- To perform high-level data querying and analytical operations using Hive, providing a simplified SQL-like interface that enables easy execution of complex analytical queries on large datasets without manually writing MapReduce code.
- To generate meaningful analytical outputs from the MovieLens dataset such as identifying top-rated movies, most active users, rating trends, and genre-based insights to understand real-world use-case applicability.
- To develop a complete Big Data workflow demonstrating end-to-end processing of data, from ingestion into HDFS to distributed computation with MapReduce, followed by structured query-based analysis through Hive.

PROBLEM STATEMENT

With the exponential growth of digital information generated from online platforms, recommendation engines, social media, and transactional applications, traditional data processing systems are no longer capable of dealing efficiently with large-scale datasets. These conventional systems lack scalability, speed, distributed processing capabilities, and the ability to handle semi-structured and unstructured data. As a result, organizations face difficulties in extracting meaningful insights from massive datasets in real time or near-real time, leading to performance bottlenecks and delays in decision-making.

The MovieLens dataset used in this project represents such a real-world Big Data scenario, containing thousands of user records including ratings, movies, genres, demographic details, and interaction patterns. Processing such a high-volume dataset using a single machine or traditional relational databases becomes slow, computationally expensive, and prone to failure due to resource limitations and lack of distributed processing. Therefore, there is a clear need for a scalable distributed approach that enables efficient storage, parallel computation, and structured analytical querying.

This project aims to address these challenges by implementing a comprehensive Big Data processing pipeline using the Apache Hadoop ecosystem. By leveraging HDFS for distributed data storage, MapReduce for parallel batch processing, and Hive for high-level SQL-based analytical querying, the project demonstrates how modern Big Data frameworks can collectively overcome the limitations of traditional systems and efficiently process and analyze large datasets to derive meaningful insights that support real-world analytical needs.

LITERATURE REVIEW

S.NO	*Paper Name*	*Author(s) & Year*	*Technologies Used*	*Research Gaps & Limitations*
1	Benchmarking Apache Spark and Hadoop MapReduce on Big Data Classification	T. Tekdogan & A. Cakmak (2022)	Hadoop MapReduce, Apache Spark	Focuses only on classification tasks; performance deteriorates for larger workloads with Spark; limited to one dataset.
2	Large-scale Data Modelling in Hive and Distributed Query Processing using MapReduce and Tez	A. Adamov (2023)	Hive, MapReduce, Tez (Hadoop ecosystem)	Examines Hive + Tez but doesn't cover Pig or Spark; limited real-world large-scale benchmarks.
3	A Comprehensive Survey of MapReduce Models for Big Data	H.B. Abdalla et al. (2025)	MapReduce (Hadoop), Hive, Pig, Spark, Cassandra	Broad survey, but lacks deep empirical experiments; survey only, not full system implementation.
4	Apache Hadoop Ecosystem	(2025) IJERT study	Hadoop Ecosystem (HDFS, Spark, etc)	Focuses on social-media analytics; limited comparison across tools beyond Hadoop + Spark.
5	An Overview of Hadoop Applications in Transportation Big Data	C. Ma et al. (2023)	Hadoop (HDFS, MapReduce) in transportation domain	omain-specific (transport) so may not generalize; doesn't deeply compare newer tools like Spark or Tez.
6	A Study, Research, and Review of the Hadoop Ecosystem	Gaurav Prajapati Yakshatha Poojari (Nov 2023)	Hadoop ecosystem broadly	Review paper; lacks experimental results or large-scale comparative benchmarks.
7	Big Data Analytics: A Comparative Evaluation of Apache Hadoop and Apache Spark	S. Singh, J. Singh, S. Singh (2023)	Hadoop, Spark	Concentrated on performance/scalability; limited in exploring query engines like Hive/Pig or export/import (Sqoop) aspects.
8	Big Data Analytics: Use of Hadoop MapReduce	J. M. Jadhav & C. V. Kulkarni (2022)	Hadoop MapReduce, HDFS	MapReduce; does not cover newer ecosystem tools like Spark or Tez.
9	Optimizations of Distributed Computing Processes on Big Data	(2023)	Hadoop ecosystem (HDFS, Hive, Pig, YARN, Spark)	Broad tool-list; less empirical benchmarking.

TECHNOLOGY STACK USED

This project utilizes essential and foundational Big Data processing and storage technologies from the Apache Hadoop ecosystem. These technologies collectively form a robust, scalable, and fault-tolerant pipeline capable of storing, processing, and analyzing large datasets efficiently. Each tool included in the stack plays a critical role in enabling distributed computation, structured querying, and the extraction of meaningful insights from real-world data.

Core Technologies:

Technology	Purpose in Project	Key Features Utilized
Hadoop (HDFS & YARN)	Distributed storage and resource management	Replication, distributed cluster management
Hadoop MapReduce	Batch processing of data and generation of analytical results	Mapper and Reducer jobs
Apache Hive	SQL-like querying and structured analytics	External tables, JOIN operations, aggregations
MySQL	External relational database integration	Reporting and structured storage
MovieLens 100k Dataset	Source dataset for analysis	rating/user/movie data files

PROJECT ARCHITECTURE

The architecture of this project represents a complete Big Data processing pipeline built using the Apache Hadoop ecosystem. It follows a layered structure where data flows from raw ingestion to processing and finally to analytical query results. The system is centered around HDFS as the distributed storage backbone, while Hadoop MapReduce and Apache Hive are responsible for performing scalable analytics using distributed and SQL-based execution models. Each layer of the architecture plays an important role in enabling data handling, fault-tolerant storage, and parallel processing, ensuring high performance and reliability while dealing with large datasets.

The architecture demonstrates how the key components of the Hadoop ecosystem interact within a practical analytical workflow. It showcases the movement of data from raw file ingestion into HDFS, through transformation via MapReduce, and finally querying using Hive, forming an end-to-end Big Data pipeline capable of handling large-scale analytical workloads.

Project Workflow:

1. Dataset Loading: The MovieLens dataset is first downloaded to the local file system. Basic preprocessing steps, such as formatting validation and column checking, are performed to ensure compatibility before uploading to Hadoop storage.

2. Ingestion to HDFS: The dataset is uploaded into HDFS using commands such as:

hdfs dfs -put <filename> /movielens

This step enables distributed storage with replication and fault tolerance. HDFS allows parallel access from multiple nodes, enabling scalable data handling.

3. Processing Using MapReduce: Custom MapReduce programs are executed to perform analytical operations such as counting movie ratings, identifying top-rated movies, and analyzing active users. The Mapper–Reducer execution model splits processing tasks into smaller subtasks that run concurrently across nodes, significantly improving computation speed for large datasets.

4. Querying with Hive: Hive provides a SQL-like query interface (HiveQL) that enables structured analytical queries without writing complex Java code. Tables are created over data stored in HDFS and analytic queries such as JOINs and aggregations are executed to derive insights. Hive internally transforms queries into MapReduce jobs, enabling scalability and performance.

DATASET DESCRIPTION

The project uses the MovieLens 100k dataset, a widely used benchmark dataset for recommendation and rating analysis systems.

Dataset Overview:

Attribute	**Description**
Dataset Size	100,000 ratings
Number of Users	943
Number of Movies	1,682
Format	CSV structured text files
Source	GroupLens Research, University of Minnesota
Files Used	u.data, u.item, u.user, u.genre, u.occupation

Key Attributes in the Dataset:

File	Fields
u.data	User ID, Movie ID, Rating, Timestamp
u.item	Movie ID, Title, Release date, Genre flags
u.user	User ID, Age, Gender, Occupation
u.genre	Movie genres
u.occupation	User job categories

Examples of Analytics Performed:

- Top 10 highest-rated movies
- Most active users (highest rating count)
- Rating distribution by age group
- Most popular movie genreAverage ratings per genre

CHALLENGES FACED

Throughout the development of the Hadoop Big Data pipeline, several technical and operational challenges were encountered across different stages of the system lifecycle, starting from environment setup to integration and performance optimization.

A. Environment & Setup Challenges: Installing Hadoop, Spark, Hive, Pig, Sqoop, and Tez with compatible versions required deep configuration knowledge. Setting up Java, SSH password-less access, Namenode formatting, and environment variables often resulted in unexpected system failures. Balancing resource allocation in a pseudo-distributed or multi-node setup required repeated tuning.

B. Data Handling & Ingestion Difficulties: The MovieLens dataset contained multiple files with inconsistent delimiters requiring preprocessing. Defining correct schemas in Hive and Pig was challenging due to missing or ambiguous fields. Commands sometimes failed due to permission issues, requiring superuser execution and ownership adjustments.

C. Processing & Execution Challenges: Debugging MapReduce code to eliminate reducer overload and null-key errors needed iterative refinement. Hive queries initially performed slowly because the default execution engine was MapReduce. Spark jobs failed due to insufficient memory, requiring executor, driver, and core configuration adjustments.

D. Integration Challenges: Sqoop import/export produced JDBC driver issues and datatype mismatches while writing into MySQL. Ensuring identical analytical results across Hive, Spark, Pig, and MapReduce caused repeated validation cycles.

E. Performance Limitations: MapReduce’s disk-based processing made it significantly slower for iterative analytics workloads.Running multiple big data tools simultaneously created CPU & RAM contention within cluster resources.

APPLICATIONS & USE CASES

This project illustrates how distributed data processing and analytics can support real-world decision-making across industries that rely on large-scale user interaction and performance insights.

A. Real-World Applications

- Recommendation Systems: Used by Netflix, YouTube, and Amazon to improve personalization by analyzing user behavior and rating patterns.
- Customer Segmentation & Marketing Analytics:Used to classify users by demographic interest and engagement level for targeted campaigns.
- Content Performance Insights:Identifying popular genres and high-rated movies assists production and licensing decisions.

B. Industry Use Cases

Industry	Example Applications
Entertainment	Personalized movie suggestions
Retail & e-commerce	User profiling & targeted advertising
Transportation	Route and traffic analytics using big data
Healthcare	Patient trend and diagnosis analytics
Finance & banking	Fraud detection using pattern mining

RESULTS

```
hive> CREATE DATABASE moviedb;
OK
Time taken: 0.522 seconds
hive> USE moviedb;
OK
Time taken: 0.04 seconds
hive> CREATE TABLE movies(
    > movie_id INT,
    > title STRING,
    > genres STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 1.523 seconds
hive> CREATE TABLE ratings(
    > user_id INT,
    > movie_id INT,
    > rating FLOAT,
    > timestamp STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
```

```
Time taken: 0.411 seconds
hive> SELECT * FROM movies LIMIT 5;
OK
1       Toy Story (1995)        01-Jan-1995
2       GoldenEye (1995)        01-Jan-1995
3       Four Rooms (1995)       01-Jan-1995
4       Get Shorty (1995)       01-Jan-1995
5       Copycat (1995)  01-Jan-1995
Time taken: 1.463 seconds, Fetched: 5 row(s)
hive> SELECT * FROM ratings LIMIT 5;
OK
196     242     3.0     881250949
186     302     3.0     891717742
22      377     1.0     878887116
244     51      2.0     880606923
166     346     1.0     886397596
Time taken: 0.163 seconds, Fetched: 5 row(s)
```

CONCLUSION

This project successfully demonstrates the development of a complete end-to-end Big Data analytics pipeline using the Apache Hadoop ecosystem, highlighting the practical integration of distributed data storage and processing technologies in a real-world analytical environment. The implementation covers the entire workflow, starting from dataset ingestion into HDFS, parallel and distributed computation using MapReduce, and high-level analytical querying using Apache Hive. By processing the MovieLens 100K dataset, the project was able to extract meaningful insights such as highly rated movies, frequently active users, rating trends, and genre-based popularity patterns. These results showcase the actual potential of Hadoop as a powerful tool for processing and analyzing large-scale datasets that traditional systems struggle to handle efficiently.

Through the execution of this project, significant learning outcomes were achieved, including hands-on understanding of distributed file storage systems like HDFS, fundamentals of parallel processing using MapReduce, and the ability to perform analytical queries using Hive's SQL-like interface. The project also strengthened technical knowledge related to cluster setup, job execution flow, fault tolerance, optimization considerations for large data operations, and handling of real-world datasets within a distributed computing environment. Practical exposure to debugging and configuration challenges added valuable problem-solving experience while working with scalable data technologies.

Overall, this work demonstrates the industrial relevance and importance of Hadoop-based Big Data frameworks in solving modern high-volume data challenges. The project proves that distributed processing can significantly improve performance and enable organizations to derive meaningful insights and make informed decisions from massive datasets. The successful implementation reinforces Hadoop's continued significance in enterprise computing and establishes a strong foundation for future enhancements such as integrating machine learning models, real-time processing, and cloud-based deployment architectures.

FUTURE SCOPE

Although the current implementation utilizes HDFS, MapReduce, and Hive to build a functional Big Data analytical pipeline, there are several opportunities to extend and improve the system for more advanced real-world applications. The following enhancements can significantly increase performance, analytical capability, automation, and practical industrial usability.

A. Enhancements & Extensions

- Integration of Machine Learning Capabilities: Future versions of the project can incorporate Spark MLlib to develop predictive models such as collaborative filtering-based recommendation systems. This would allow the system to predict user ratings and provide intelligent movie suggestions rather than focusing only on analytical aggregation.
- Real-Time Stream Processing: Implementing technologies such as Apache Kafka, Apache Flink, or Spark Streaming would enable real-time capture and processing of continuously incoming data, similar to platforms like Netflix or YouTube that analyze user interactions instantaneously.

B. Workflow Automation & Visualization

- Pipeline Orchestration: Tools like Apache Airflow or Apache Oozie can be integrated to schedule automated workflows, manage dependencies, restart failed jobs, and monitor periodic execution pipelines efficiently.
- Dashboard-Based Visualization: Analytical results can be represented visually using Tableau, Power BI, Grafana, or Apache Superset, enabling interactive dashboards for business reporting rather than static query outputs.

C. Inclusion of Additional Hadoop Ecosystem Tools

- Apache Spark — for faster in-memory processing and iterative analytics
- Apache Pig — for script-based ETL data transformation
- Apache Tez — for DAG-based execution and improved Hive performance
- Apache Sqoop — for importing/exporting data to relational databases such as MySQL or PostgreSQL

REFERENCES

[1] "An Image Colorization Approach based on GAN and Classifier," arXiv preprint arXiv:2209.10637, Sep. 2022.

[2] "arXiv preprint arXiv:2301.12454, Jan. 2023.

[3] Data, vol. 9, no. 4, art. 77, 2024.

[4] "Apache Hadoop Ecosystem," *International Journal of Engineering Research & Technology (IJERT)*, 20XX.

[5] The Journal*, vol. XX, no. YY, pp. ZZ-AA, 2023.

[6] International Journal of Novel Research and Development (IJNRD)*, vol. X, no. Y, 2023.

[7] International Research Journal of Engineering & Technology (IRJET)*, vol. 10, no. 11, 2023.

[8] London Journal of Research in Computer Science & Technology (LJRCST)*, 23